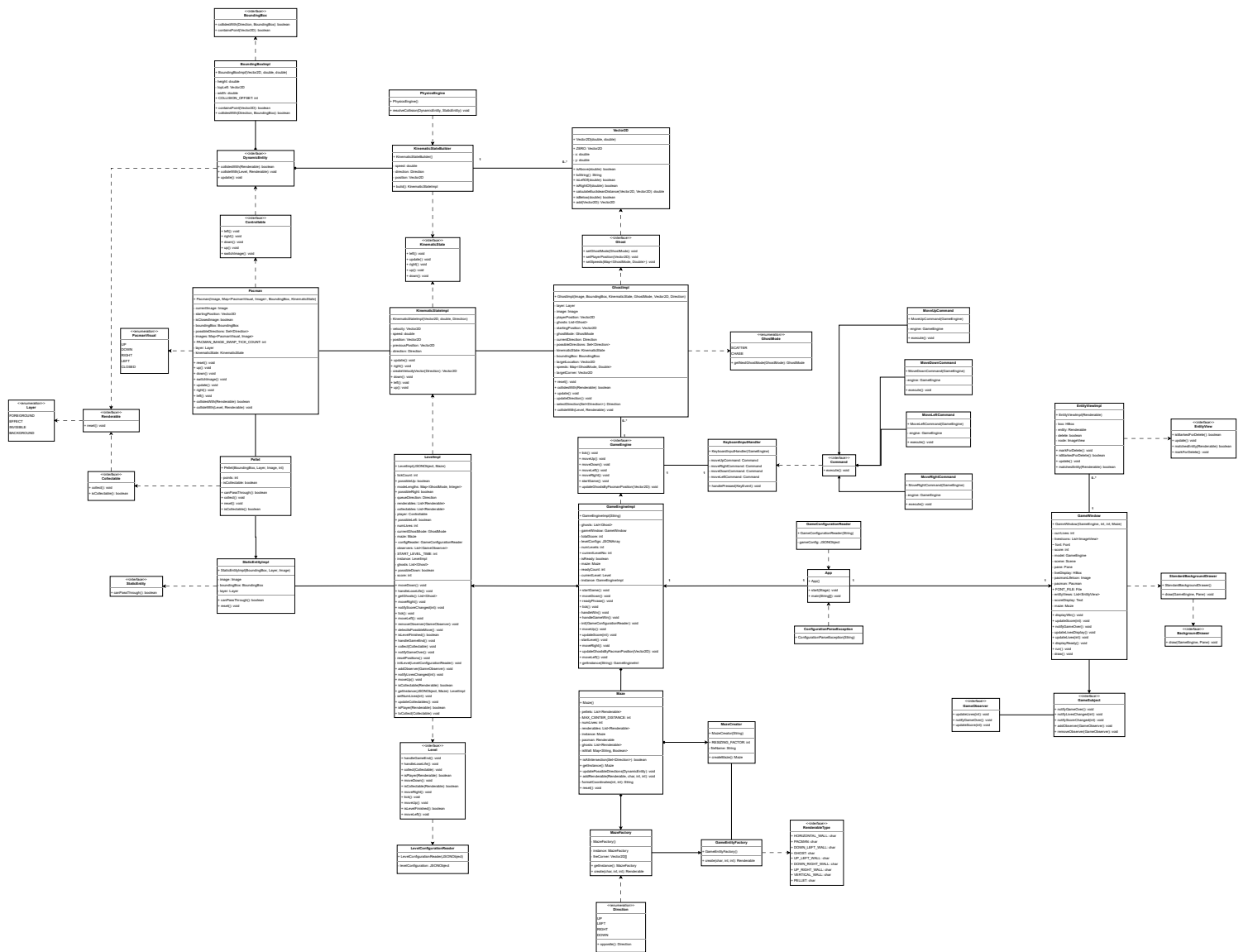




Design Overview

The Pacman game involves a series of design patterns, including Observer Pattern, Factory Method Pattern, Singleton Pattern and Command Pattern. The target of the pattern implementation is to concatenate major classes to make Pacman work properly.

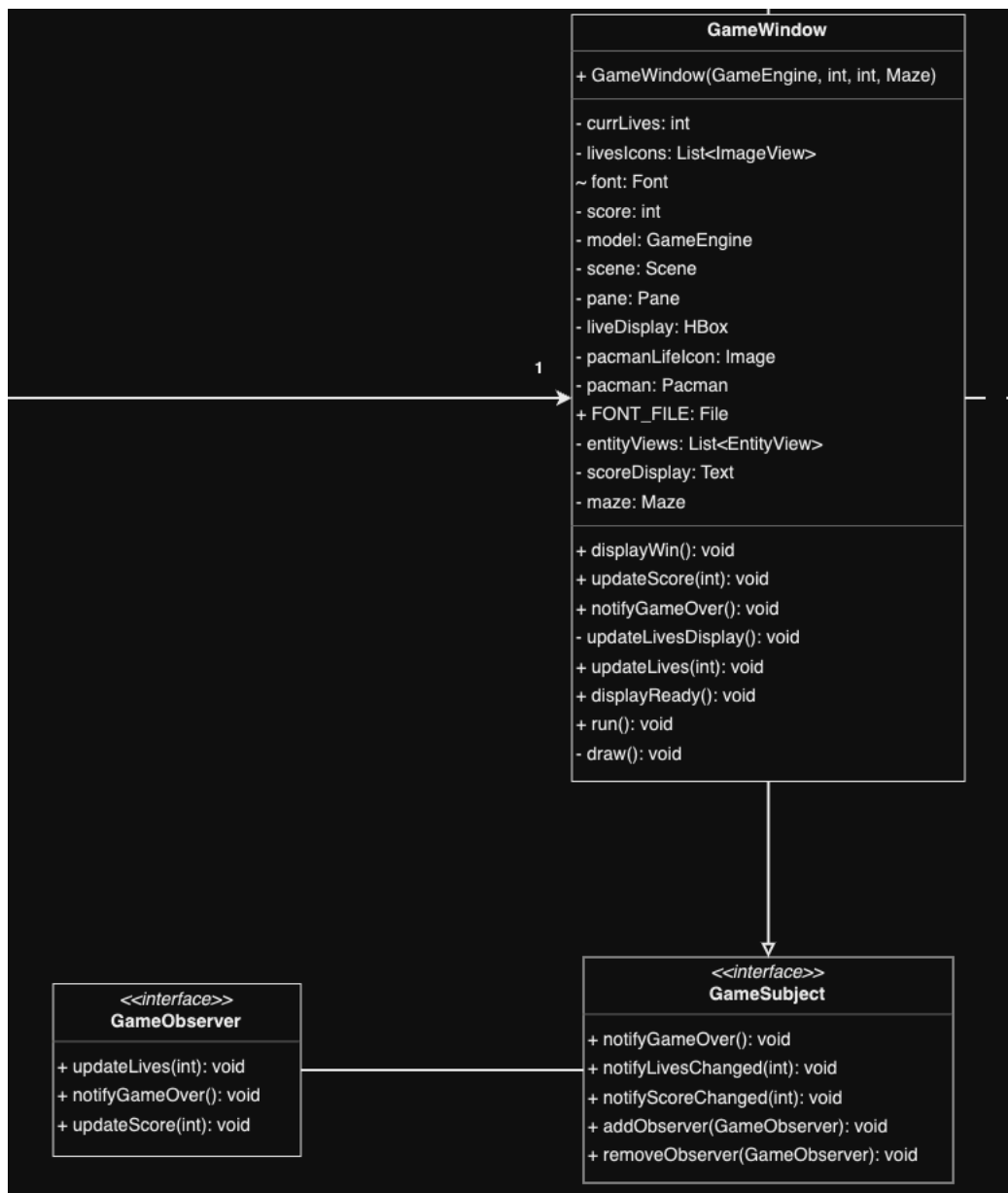
Design Patterns

As the previous UML diagram shown, each of the four patterns creates related classes (interfaces) to allow the other main classes (like App, GameEngineImpl, GameWindow, LevelImpl, etc.) to do their jobs and keep the code readable.

Involved in specific classes and UML diagram

Observer Pattern

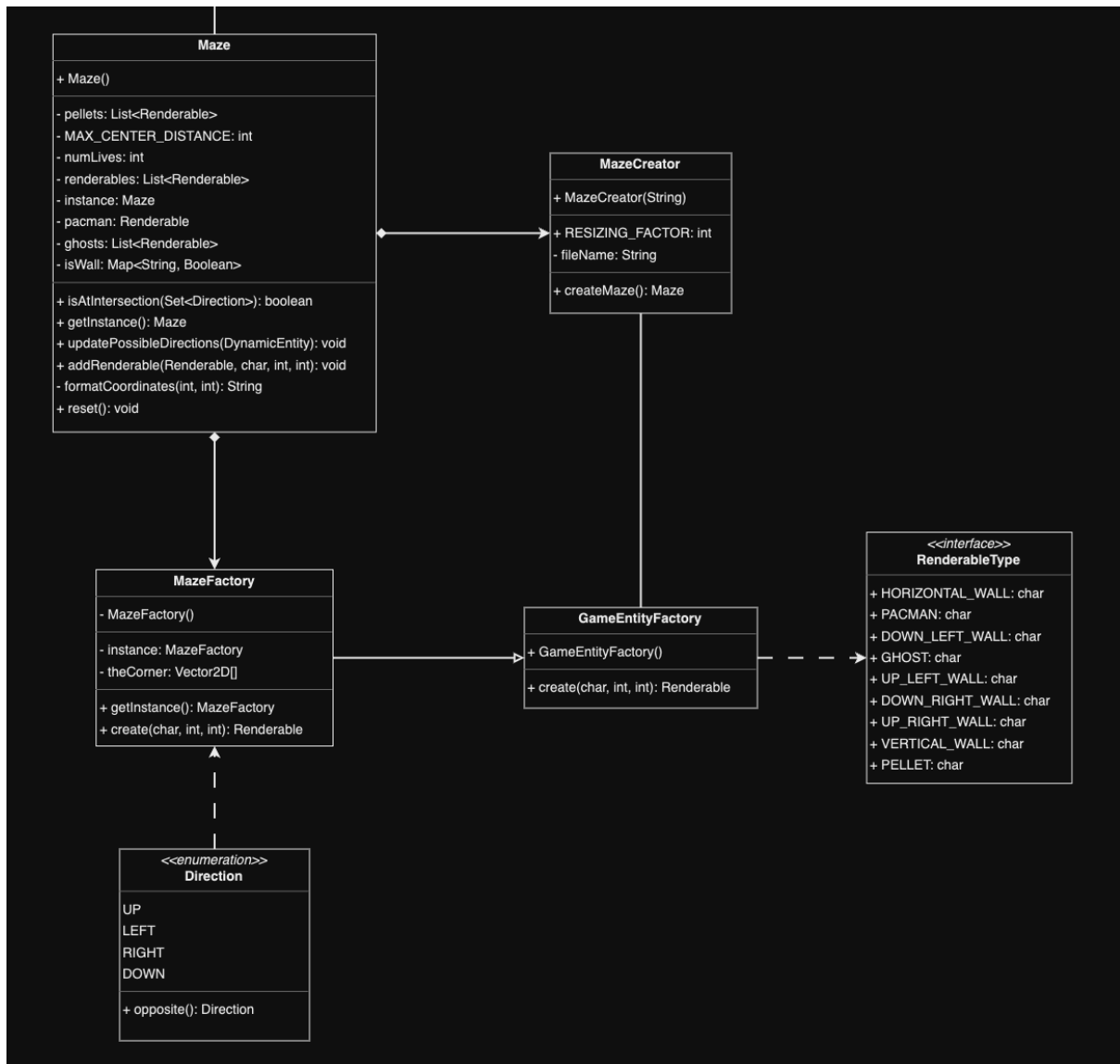
- Classes involved:
 - GameObserver: Methods to update UI based on game changes (lives and score changes)
 - GameSubject: Manage the observers and notify changes to them
 - GameWindow: Implement GameObserver to receive updates and then change UI
 - LevelImpl: Implement GameSubject to notify the changes to observers
- Linked to UML diagram:
 - An association between GameSubject and GameObserver, where GameSubject contains a range of observers.
 - A realisation relationship from GameWindow to GameSubject.



Factory Method Pattern

- Classes involved:
 - o **GameEntityFactory**: Declare the factory method `create()` for game entities creation
 - o **MazeFactory**: Concrete class of **GameEntityFactory** that implements `create()`
 - o **RenderableType**: Enumeration class includes different types of game objects
- Linked to UML diagram:
 - o A composition relationship from **Maze** to **MazeFactory**.
 - o A composition relationship from **Maze** to **MazeCreator**.
 - o A dependency relationship from **Direction** to **MazeFactory**.

- An inheritance relationship between GameEntityFactory and MazeFactory.
- An association between GameEntityFactory and MazeCreator.

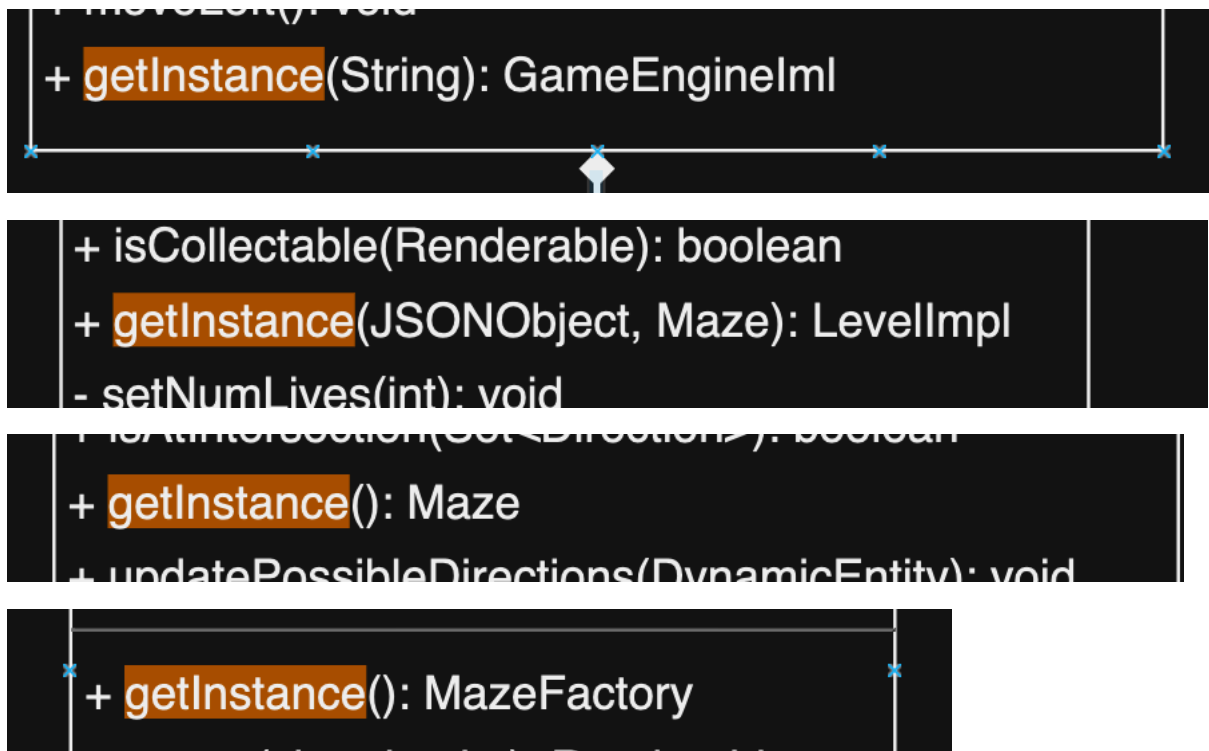


Singleton Pattern

The Singleton Pattern is used to ensure there is only one instance of the implemented class exists throughout the game lifecycle.

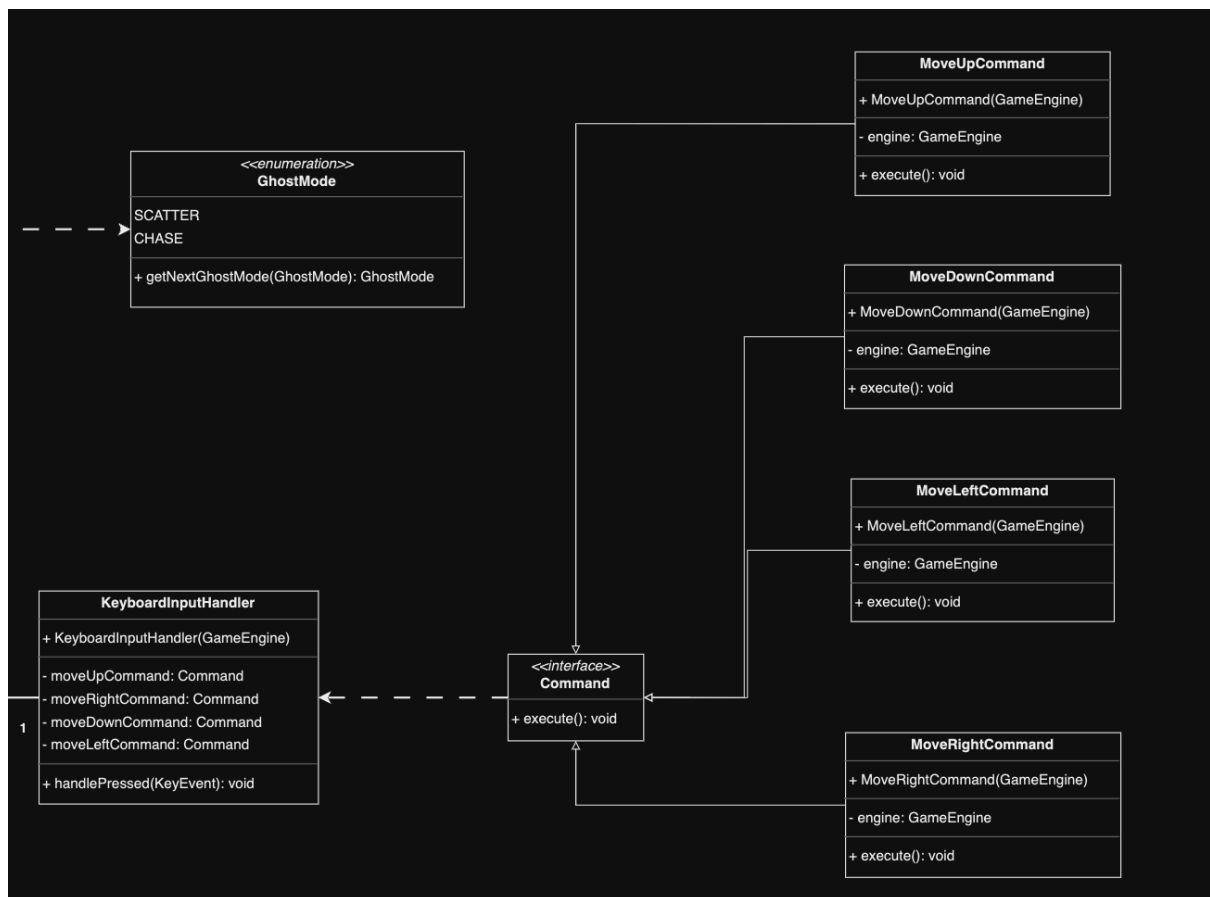
- Classes involved:
 - GameEngineImpl
 - LevelImpl
 - Maze
 - MazeFactory
- Linked to UML diagram:
 - A private constructor on each Singleton Pattern

- A static method getInstance() in each Singleton class controls access to the single instance.



Command Pattern

- Classes involved:
 - Command: Declare execute() to be implemented by the concrete command classes
 - KeyboardInputHandler: Manage player inputs and direct PacMan with the appropriate commands
 - Concrete command classes: Implement Command to do specific movements
- Linked to UML diagram:
 - Concrete commands (MoveUpCommand, MoveDownCommand, MoveLeftCommand, MoveRightCommand) inherits to Command.
 - A dependency relationship from Command to KeyboardInputHandler.



Benefits

1. Decoupling the game logic from UI design

In Java Gradle game program writing, App and GameWindow classes refer more specifically to writing front-end UI designs. The use of Observer design Patterns rounds out this requirement, preventing developers from confusing class functionality and affecting readability.

As the following code shows (copied from LevelImpl), LevelImpl does not need to know how the UI is implemented or updated. And just modify the score in GameWindow.

```

public class LevelImpl implements Level, GameSubject {
    private List<GameObserver> observers;
    @Override
    public void notifyLivesChanged(int lives) {
        // update live changes by implementing observer method
        for (GameObserver observer : observers) {
            observer.updateLives(lives);
        }
    }
}

```

2. Single Responsibility Principle (SRP) achieved

Each class created by the factory method pattern is responsible for creating a specific type of game entity. The use of this pattern reduces the responsibility of game logic classes (e.g., multiple methods stacked in the same class) and eliminates the need for classes to know the details of how entities are created. This makes it easier to maintain and extend to meet the single-responsibility principle.

As this code shows (copied from MazeFactory), MazeFactory is only responsible for creating various game entities.

```
public class MazeFactory extends GameEntityFactory {  
    public Renderable create(char renderableType, int x, int y) {  
        switch (renderableType) {  
            case RenderableType.PACMAN:  
                return new Pacman(...);  
            case RenderableType.PELLET:  
                return new Pellet(...);  
            case RenderableType.GHOST:  
                return new GhostImpl(...);  
            default:  
                return null;  
        }  
    }  
}
```

3. Encapsulation of user actions

Command mode encapsulates user operations into separate command objects, realizing the separation of operations from the actual logic. This effectively improves the flexibility of the code and does not affect the handling of keyboard input in subsequent extensions.

The code below encapsulates the logic for moving Pacman down and shows that this refers to entering keyboard input into Pacman for processing.

```

public class MoveDownCommand implements Command {
    private GameEngine engine;
    public MoveDownCommand(GameEngine engine){
        this.engine = engine;
    }

    @Override
    public void execute() {
        engine.moveDown();
    }
}

```

4. Liskov Substitution Principle implementation

With Singleton creation, this helps keep the game mechanics consistent and avoids errors (such as level-crossing) that can occur when managing multiple levels at the same time.

As for the single instance, it is mainly used in the main control class, which can better handle the infinite creation of constructors in multiple runs of the game resulting in the values being refreshed each time, thus effectively avoiding the game body from getting stuck.

```

public static LevelImpl getInstance(JSONObject levelConfiguration, Maze maze) {
    instance = new LevelImpl(levelConfiguration, maze);
    return instance;
}

```

Drawbacks

One of the weaknesses that this project needs to maintain the observers and notifications for handling different UI messages. Therefore, the working criteria increases the complexity of code.

SOLID/GRASP principles

SOILD

- Single Responsibility Principle (SRP): Factory method ensures the consistent of entity creations.
- Open/Closed Principle (OCP): Factory method allows adding entities through the factory extension, leading the open extensions and closed modifications.

- Liskov Substitution Principle (LSP): Singleton pattern allows only one instance exists of the classes.
- Interface Segregation Principle (ISP): Command pattern keeps the interface minimal and focused.
- Dependency Inversion Principle (DIP): The game depends on the command pattern not other specific one.

GRASP

- Controller: Singleton patterns ensure only one centralised controller handling.
- Low Coupling: Observer pattern ensures the game engine only updates to the observers.
- High Cohesion: Factory pattern ensures classes only focused on Maze entity.
- Polymorphism: Different command patterns provide own logic to deal with keyboard inputs.
- Protected Variations: Factory method protects game entities not influenced by other engines.

Assignment 1 reflection

The use of design patterns was not considered in Assignment 1. This resulted in a complex and awkward initial design. This update, however, especially how the behavior of ghosts is handled through the GhostBehavior interface, made the design more flexible and easier to maintain. In addition, the problems of fewer abstractions and too much reliance on inheritance have been enhanced with the introduction of more patterns. This has resulted in increased modularity and reduced code duplication. These design changes have helped streamline the development process, especially in managing levels and game progress.

Change Justification

The main improvements based on Assignment 1 involved the Singleton pattern for key entity classes (GameEngineImpl, LevelImpl, etc.) to share data more validity. And the Factory method pattern is added to declare the different game entities creation to achieve LSP. Also, the player controllable provides more information on Pacman movement. Furthermore, the UI design is enhanced by Observer pattern for updating and notifying. These enhancements increase Pacman game's modularity and maintainability.